Agentic AI

END TO END

1. Prompting for Effective LLM Reasoning and Planning

Introduction to Prompting for Effective LLM Reasoning and Planning

Prerequisites for the Course

- **Core Technical Foundations**
  - Proficiency in **Python** (syntax, data structures, functions, scripting)
  - Familiarity with **Jupyter Notebook** (executing cells, analyzing outputs)
  - Conceptual understanding of **Large Language Models (LLMs)** and prompting
- **AI & Agentic Concepts**
  - Fundamentals of **AI Agents** (reasoning, planning, environment interaction)
  - Understanding **advanced prompting techniques** for precise control
  - Role-based prompting for creating **context-aware AI personas**
- **Reasoning & Problem-Solving Techniques**
  - **Chain-of-Thought (CoT):** Enhancing step-by-step reasoning
  - **ReAct (Reason + Act):** Enabling LLMs to use tools and perform actions
- **Workflow & Optimization Skills**
  - **Prompt refinement:** Iterative improvement for better outputs
  - **Prompt chaining:** Designing multi-step AI workflows
  - **Feedback loops:** Enabling AI systems to self-improve over iterations

The Role of prompting Agentic AI with Python and OpenAI

What is an AI Agent?

- **Definition**
  - An **AI Agent** is an intelligent system that can **perceive its environment, make decisions, and take actions** to achieve specific goals.
- **Core Characteristics**
- **Autonomy:** Operates independently with minimal human intervention
  - **Adaptability:** Adjusts behavior based on changing inputs or feedback
  - **Interaction:** Engages with environments (digital or physical)
- **How It Differs from Traditional AI**
  - Moves beyond **static rules and simple outputs**
  - Leverages **LLMs** for deeper reasoning and understanding
  - Capable of **goal-driven behavior** instead of one-time responses
- **Key Capabilities**
  - **Reasoning:** Analyzes problems step-by-step
  - **Planning:** Breaks tasks into actionable steps
  - **Action:** Executes tasks using tools or APIs
  - **Observation:** Learns from outcomes to improve next steps
- **Key Takeaway**
  - AI Agents transform LLMs from passive responders into **active problem-solvers** that can **reason, plan, and act autonomously** in real-world environments.

Components of an AI Agent

- **1. Large Language Model (LLM) – The Brain**
  - Enables **understanding, reasoning, and decision-making**
  - Processes and generates language for intelligent behavior
- **2. Tools – Action Enablers**
  - External **APIs, functions, and systems**
  - Allow agents to **search, calculate, query data, and interact with environments**
- **3. Instructions – Behavior Guide**
  - Defined via **system prompts or rules**
  - Direct how the agent **acts, responds, and makes decisions**
- **4. Memory – Context & Learning**
  - **Short-term:** Current conversation context
  - **Long-term:** Past interactions and learned patterns
- **5. Runtime / Orchestration Layer – The Controller**
  - Manages **execution flow and decision-making**
  - Decides **when and how to use tools**
  - Executes actions since LLMs themselves only generate text

Why Build with AI Agents?

- **Limitations of Standalone LLMs**
  - Struggle with **complex, multi-step tasks**
  - Lack **real-time data access** beyond training
  - Limited ability for **planning and execution**
- **How Agents Solve This**
  - Combine **LLM reasoning + tools + planning**
  - Enable **multi-step workflows** and task execution
  - Access **real-time data and external systems (APIs)**
- **Real-World Capabilities**
  - Perform **end-to-end task automation**
  - Interact with **digital and physical environments**
  - Execute actions beyond text generation

What is Prompting?

- **Definition**
  - **Prompting** is the process of providing **structured instructions** to a Large Language Model (LLM) to guide its behavior, responses, and outputs.
- **Role in AI Systems**
  - Acts as a way to **"program" LLMs without code**
  - Shapes how the model **understands, reasons, and responds**
  - Enhances and refines the model's capabilities
- **Key Components of a Prompt**
  - **System Prompt:** Defines overall behavior and rules
  - **User Input:** Specifies the task or query
  - **Tool Descriptions:** Guide how and when tools are used (in agents)
- **Why Prompting Matters**
  - Determines **output quality and accuracy**
  - Enables **control, customization, and consistency**
  - Essential for building **reliable AI agents and workflows**

Real-World Use Case – Processing a Refund

- **Scenario**
  - A customer submits a **refund request**, which acts as the initial input to an AI agent.
- **Agent-Driven Approach**
  - Instead of rigid scripts, the agent uses **LLM-powered reasoning and planning**
  - Breaks down the request into **smaller, actionable steps**
- **Step-by-Step Workflow**
  1. **Verify Purchase History** – Confirm the transaction
  2. **Check Refund Policy** – Validate eligibility
  3. **Assess Request Reason** – Understand customer intent
  4. **Decide Next Action** – Approve, reject, or escalate
- **Key Concept**
  - Uses **planning + prompting techniques** to decompose complex tasks
  - Executes a **multi-step, goal-driven workflow**

Role-Based Prompting:

Introduction to Role-Based Prompting

- **What is Role-Based Prompting?**
  - Role-Based Prompting is the technique of assigning a **specific persona or identity** to an LLM to generate **specialized and context-aware responses**.
- **Why It Matters**
  - Default LLM responses are often **generic**
  - Personas enable **expert-level, creative, or stylistic outputs**
  - Improves **consistency, tone, and relevance**
- **How It Works**
  - Define a **role/persona** in the prompt
  - Provide **behavior, tone, and perspective guidelines**
  - Model adapts responses to match the assigned identity
- **Example**
  - **Generic:** "Cats are common pets…"
  - **Literary Persona:** "Cats are celebrated for their playful elegance…"
  - **Pirate Persona:** "Ar, cats be clever mates with a mischievous spirit!"
- **Key Concept**
  - A **Persona** shapes how the AI **thinks, speaks, and responds**, similar to assigning a role to an actor.

Why is Role-Based Prompting Effective?

- **Core Reason**
  - LLMs are trained on **diverse data and multiple styles**, but they require guidance to produce **focused, task-specific outputs**.
- **How Role-Based Prompting Helps**
  - Directs the model to adopt a **specific tone, style, and perspective**
  - Aligns responses with **task requirements and user intent**
  - Reduces ambiguity in how the model should respond
- **Without a Defined Role**
  - Output may be **inconsistent or unfocused**
  - Style can vary (formal, casual, humorous, etc.)
  - Less **precision and relevance**
- **With a Defined Role**
  - Produces **targeted and consistent responses**
  - Ensures **clarity in tone and communication style**
  - Enhances **quality and reliability of outputs**

Crafting a Good Role-Based Prompt

- **Overview**
  - Effective role-based prompting requires **clear definition of the persona and task**, ensuring outputs are **consistent, relevant, and high-quality**.
- **Key Components**
  - **[Role]:** Define the persona (e.g., *"Act as a pirate"*)
  - **[Task]:** Specify the exact instruction or query
  - **[Output Format]:** Describe how the response should be structured
  - **[Examples]:** Provide sample inputs/outputs for guidance
  - **[Context]:** Include additional relevant information
- **Best Practices**
  - Be **specific about personality, tone, and expertise**
  - Clearly define **expectations and constraints**
  - Use examples to **guide model behavior effectively**

Evaluating Agent Personas

- **Why Evaluation Matters**
  - Ensures outputs are **accurate, consistent, and aligned with expectations**
  - Similar to a **director reviewing an actor's performance**
- **Ground Truth Evaluation Approach**
  - Define **success criteria** (accuracy, tone, format)
  - Create **test inputs + ideal outputs (ground truth)**
  - Compare agent responses against **expected results**
- **Evaluation Techniques**
- **Structured Outputs (e.g., JSON):**
  - Validate format and values
  - Use metrics like **accuracy, precision, recall, completeness**
- **Unstructured Outputs (text/images):**
  - Use **LLM-as-a-Judge** to score responses
  - Compare output with ground truth based on defined criteria
- **Best Practice**
  - Analyze **raw inputs & outputs (traces)**
  - Identify hidden errors and improve system behavior

Other Evaluation Types for AI Agents

- **Beyond Ground Truth Evaluation**
  - Additional evaluation methods help ensure **robustness, reliability, and real-world performance**.
- **Key Evaluation Methods**
- **Consistency & Persona Adherence:**
  - Checks if the agent stays within its **defined role and behavior**
  - Can be evaluated using **mock conversations + LLM-as-a-judge**
- **Robustness Testing:**
  - Tests resilience against **adversarial or unexpected inputs**
  - Critical for **safety-sensitive applications**
- **Simple Metrics:**
  - Track measures like **response length, latency, and cost**
  - Useful for **usability and efficiency monitoring**
- **Continuous Monitoring**
  - Evaluate during **development and production**
  - Analyze performance with **real-world data streams**
- **Iterative Improvement**
  - Refine prompts based on **evaluation feedback**
  - Add clarity, constraints, or examples for better results
  - Treat AI systems like **software: test → measure → improve**

Applying Role-Based Prompts Across Domains

- **Overview**
  - Role-based prompting enables AI to deliver **domain-specific, high-quality outputs** by combining **persona, task, format, and examples**.
- **Coding – Senior Python Developer**
  - **Role:** Expert in efficient algorithms & clean code (PEP 8, type hints)
  - **Task:** Build a memoized Fibonacci function
  - **Outcome:** Produces **robust, optimized, and well-documented code**
- **Data Analysis – Marketing Analyst**
  - **Role:** Experienced in sales data & strategy
  - **Task:** Analyze sales trends and generate insights
  - **Format:** Bullet points with **Findings + Recommendations**
  - **Outcome: Actionable business insights** for decision-making
- **3. Creative Writing / Other Domains**
  - **Role:** Defines tone, style, and perspective
  - **Task:** Content generation (story, script, etc.)
  - **Outcome: Engaging, context-aware, and stylistically consistent outputs**

Chain-of-Thought and ReACT Prompting:

Introducing Chain-of-Thought & ReAct Prompting

- **The Challenge with LLMs**
  - Limited access to **real-time or proprietary data**
  - Operate as **passive systems (input → output)**
  - Struggle with **complex, multi-step problem solving**
- **The Solution: Advanced Prompting Techniques**
- **Chain-of-Thought (CoT):**
  - Guides the model to **think step-by-step**
  - Improves reasoning and problem decomposition
- **ReAct (Reason + Act):**
  - Combines **reasoning with actions (tool usage)**
  - Enables interaction with **external systems and data**
- **Why These Matter**
  - Transform LLMs from **text generators → problem solvers**
  - Enable handling of **complex, real-world tasks**
  - Bridge the gap between **thinking and doing**
- **Analogy**
  - Like a **brilliant but isolated scholar**:
  - Great at knowledge-based answers
  - Struggles with **real-time, interactive tasks** without guidance or tools

Chain-of-Thought (CoT) Prompting

- **What is CoT?**
  - Chain-of-Thought (CoT) is a prompting technique that encourages LLMs to **generate step-by-step reasoning** before arriving at a final answer.
- **Why It Matters**
  - Improves **accuracy and logical reasoning**
  - Breaks complex problems into **manageable steps**
  - Enhances **transparency of the model's thinking process**
- **Types of CoT**
  - **Zero-Shot CoT:**
    - Add simple instruction like *"Let's think step by step"*
    - No examples required
  - **Few-Shot CoT:**
    - Provide **examples with reasoning + final answers**
  - Helps model learn **how to think, not just what to answer**
- **Example Use Case**
  - Instead of: *"Is 3 + 1 = 4 correct?"*
  - Use: *"Solve step-by-step, then compare with the given answer."*

Benefits of Chain-of-Thought (CoT) Prompting

- **1. Improved Performance**
  - Enhances accuracy in **reasoning tasks** (arithmetic, logic, commonsense)
  - Breaks problems into steps, reducing **errors and hallucinations**
- **2. Interpretability & Trust**
  - Makes reasoning **transparent and traceable**
  - Helps in **debugging and validation** of outputs
  - Transforms LLMs from a **black box → explainable system**
- **3. Advanced Techniques**
  - **Self-Consistency Prompting:**
  - Generates multiple reasoning paths
  - Selects the **most consistent/frequent answer**

Introducing ReAct (Reason + Act) Prompting

- **What is ReAct?**
  - ReAct is a prompting technique that combines **reasoning + action**, enabling LLMs to **interact with tools and external environments** while solving tasks.
- **Why ReAct is Needed**
  - CoT handles **internal reasoning**, but lacks real-world interaction
  - ReAct enables **dynamic problem-solving with real-time data and tools**
- **Core ReAct Loop**
  - **Thought → Action → Observation (Repeat)**
  - **Thought:**
    - Plan the next step using reasoning
  - **Action:**
    - Invoke a **tool/API** (search, calculator, database, etc.)
  - **Observation:**
    - Receive results and update understanding
- **How It Works**
  - Iteratively **reasons, acts, and learns from feedback**
  - Uses tools via an **orchestration layer**
  - Continues until the **final goal is achieved**

Designing ReAct Prompts

- **Overview**
  - Designing ReAct prompts requires structuring interactions to follow a **Thought → Action → Observation loop**, enabling reasoning with tool usage.
- **Key Design Principles**
- Instruct the model to output:
  - **Thought:** Reasoning step
  - **Action:** Tool call (often in structured format like JSON)
- Feed **Observation (tool result)** back into the model
- Maintain **message history** for context across iterations
- Use an **orchestrator** to:
  - Execute tools
  - Manage interactions
  - Control loop termination
- Example FlowSystem:
- Define tools & response formatUser: “What is the weather today?”Assistant:Thought: Need weather dataAction: Call get_weatherObservation: “Sunny, 30°C”Assistant:Thought: Ready to respondAction: final_answer("Sunny, 30°C")